Functional Requirements Document (FRD) — WariMitra

Document Identifier: WM-FRD-2026-V2.0

Classification: Detailed Functional Specification & Behavioral Contracts

Standard: IEEE Std 1233-1998 / BABOK v3 Chapter 7

Target Audience: Engineering Leads, QA Automation Engineers, Technical Business Analysts, Solution Architects

Source Code Basis: `functions/src/index.ts`, `frontend/src/app/`, `mobile/lib/screens/`, `firestore.rules`

1. Document Overview & Functional Scope

This Functional Requirements Document details the behavioral specifications, inputs, outputs, preconditions, exception paths, and Given-When-Then acceptance criteria for all production-grade features in WariMitra.

2. Feature Specification 1: Emergency SOS Trigger & Idempotent Ingestion

2.1 Metadata & Description

- **Requirement ID:** `FR-SOS-001`
- **Module:** Emergency Management Subsystem
- **Feature:** 3-Second Hold SOS Panic Trigger with Idempotency & Rate Limiting
- **Description:** Provides an infallible mobile distress mechanism that captures precise device GPS coordinates, checks against client idempotency keys, enforces rate limits, validates geographical bounds, and pushes alerts to responders.
- **Primary Actor:** Pilgrim / Devotee (`pilgrim`)
- **Secondary Actor:** Police Officer, Medical Officer, System Cloud Functions

2.2 Preconditions & Triggers

- **Preconditions:**
  1. Pilgrim is authenticated via Firebase Auth (`context.auth != null`).
  2. Device location permissions (GPS) are granted.

- **Trigger:** User presses and continuously holds the red SOS circular button on the mobile UI for 3.0 seconds.

2.3 Detailed Execution Flows

Main Success Flow (Happy Path)

1. User holds the SOS button on `sos_screen.dart`.
2. UI displays a radial progress indicator and executes a physics-based haptic vibration pulse.
3. At exactly 3.00 seconds, the client calls `geolocator.getCurrentPosition()`.
4. Client generates a version-4 UUID `idempotencyKey`.
5. Mobile client calls Firebase HTTPS Callable Function `createSOS({ latitude, longitude, severity: 'critical', idempotencyKey })`.
6. Function validates user authentication token.
7. Function queries `sosAlerts` for an existing document with the identical `idempotencyKey`. None is found.
8. Function checks recent alerts created by this `userId` in the past 60,000 milliseconds. Count is < 5.
9. Function verifies that `latitude` $\in [8.0, 35.0]$ and `longitude` $\in [68.0, 97.5]$ (India bounds).
10. Function computes a 9-character `geohash` from coordinates.
11. Function creates a document in `/sosAlerts/` with fields: `userId`, `status: 'pending'`, `latitude`, `longitude`, `geohash`, `severity`, `idempotencyKey`, `createdAt: serverTimestamp()`.
12. Function writes an immutable audit record to `/auditLogs/`.
13. Function returns `{ alertId: docRef.id, status: 'pending', message: 'SOS created successfully' }`.
14. Mobile UI displays green confirmation screen with ETA counter and emergency hotline direct dial buttons (108/100).

Alternative Flow A: Network Retry / Duplicate Idempotent Submission

- **A.1:** Mobile app experiences cellular packet loss after writing to server and re-invokes `createSOS` with the same `idempotencyKey`.
- **A.2:** Function identifies existing document with matching `idempotencyKey`.
- **A.3:** Function immediately returns `{ alertId: existing.id, status: existing.status, message: 'SOS already exists (duplicate prevented)' }`.

- **A.4:** System prevents duplicate emergency dispatches and alert fatigue.

Exception Flow 1: Rate Limit Exceeded

- **E.1:** Malicious user or glitching device attempts to send > 5 alerts within 60 seconds.
- **E.2:** Function detects `recentAlerts.size >= 5`.
- **E.3:** Function throws `https.HttpsError('resource-exhausted', 'Rate limit exceeded: Max 5 SOS per minute')`.
- **E.4:** Client displays warning in Marathi: *"कृपया प्रतीक्षा करा. एका मिनिटात कमाल ५ विनंत्या पाठवल्या जाऊ शकतात."*

Exception Flow 2: Coordinates Outside India Geofence

- **E.1:** Device transmits spoofed or invalid coordinates (e.g., Lat: 51.5074, Lon: -0.1278).
- **E.2:** Function evaluates `isInIndia` to `false`.
- **E.3:** Function throws `https.HttpsError('failed-precondition', 'Location outside India bounds')`.
- **E.4:** Request is terminated and logged as an anomaly.

2.4 Inputs & Outputs

- **Inputs:** `latitude` (Float), `longitude` (Float), `severity` (String: 'low'|'medium'|'high'|'critical'), `idempotencyKey` (UUIDv4 String), `description` (Optional String).
- **Outputs:** `alertId` (Firestore Document ID), `status` (String: 'pending'), `message` (String).

2.5 Acceptance Criteria (Given-When-Then)

```
Scenario: Successful emergency SOS creation under normal conditions
  Given an authenticated pilgrim with GPS coordinates 18.5204° N, 73.8567° E
  And the pilgrim has created fewer than 5 alerts in the last 60 seconds
  When the pilgrim holds the SOS button continuously for 3.0 seconds
  Then the createSOS Cloud Function shall create a new document in sosAlerts with status "pending"
  And the response time shall be under 2.0 seconds
  And an immutable auditLog document shall be created with eventType "SOS_CREATED"

Scenario: Prevention of duplicate dispatch via idempotency
  Given an existing SOS alert created with idempotencyKey "uuid-1234-abcd"
  When another createSOS request arrives with identical idempotencyKey "uuid-1234-abcd"
  Then no new document shall be inserted into sosAlerts
  And the function shall return the existing alertId with status "pending"
```

3. Feature Specification 2: Geospatial Medical Camp Discovery

3.1 Metadata & Description

- **Requirement ID:** `FR-MED-002`
- **Module:** Healthcare & Triage Subsystem
- **Feature:** Proximity-Based Medical Camp Radar & Capacity Query
- **Description:** Executes Geohash-based geospatial bounding queries on Cloud Firestore to return active medical relief camps within a dynamic radius (default 10 km), sorted by Haversine distance.
- **Primary Actor:** Pilgrim, Volunteer, EMT Responder
- **Implementation Source:** `functions/src/index.ts:122-177`

3.2 Preconditions & Triggers

- **Preconditions:** User is authenticated; device GPS location is available.
- **Trigger:** User navigates to Medical Camps screen in mobile app or web portal.

3.3 Execution Flow & Business Logic

1. Client acquires device `latitude` and `longitude`.
2. Client calls HTTPS Callable Function `getMedicalCamps({ latitude, longitude, radiusKm: 10 })`.
3. Function computes Geohash bounding boxes for center coordinates and radius in meters ($10 \times 1000 = 10{,}000\text{m}$) using `geofire-common.geohashQueryBounds()`.
4. Function executes parallel Firestore queries on `medicalCamps` where `status == 'active'` within each bounding box.
5. Function calculates exact Great-Circle distance (`distanceBetween`) between user coordinates and each returned camp.
6. Function filters out camps with `distanceInKm > radiusKm`.
7. Function sorts remaining camps in ascending order of `distanceKm`.
8. Function returns JSON payload: `{ camps: [ { id, name, latitude, longitude, capacity, currentPatients, staffCount, distanceKm } ] }`.
9. Client renders map markers and cards displaying camp name, distance (e.g., "1.4 km"), and available bed indicator (`capacity - currentPatients`).

3.4 Acceptance Criteria (Given-When-Then)

```
Scenario: Querying active medical camps within 10 km
  Given user is located at Saswad (Lat: 18.3448° N, Lon: 74.0298° E)
  And active medical camps exist at 2.1 km, 8.5 km, and 14.0 km
  When the user calls getMedicalCamps with radiusKm = 10
  Then the returned camps array shall contain exactly the 2.1 km and 8.5 km camps
  And the 14.0 km camp shall be excluded
  And the results shall be sorted with the 2.1 km camp at index 0
```

4. Feature Specification 3: Command Center Virtualized Incident Triage

4.1 Metadata & Description

- **Requirement ID:** `FR-CMD-003`
- **Module:** Command Center & GIS Subsystem
- **Feature:** High-Throughput Real-Time Incident Feed & Map Clustering
- **Description:** Provides police and government commanders with an auto-updating real-time incident queue utilizing `react-window` virtualization and Google Maps marker clustering to prevent browser freeze under 1,000+ simultaneous alerts.
- **Primary Actor:** Police Officer (`police_officer`), District Administrator (`admin`)
- **Implementation Source:** `frontend/src/app/sos/page.tsx`, `frontend/src/app/page.tsx`, `frontend/package.json:33`

4.2 Preconditions & Triggers

- **Preconditions:** Commander is authenticated with role `police_officer` or `admin`.
- **Trigger:** Dashboard loads and establishes Firestore real-time snapshot listener on `sosAlerts` collection ordered by `createdAt DESC`.

4.3 Execution Flow & Business Logic

1. Web client mounts `useEffect` hook subscribing to `onSnapshot(query(collection(db, 'sosAlerts'), orderBy('createdAt', 'desc')))`.
2. When a new document arrives, state updates reactively without page refresh.
3. Incidents are passed to `FixedSizeList` from `react-window` to render only visible DOM nodes,

3. Incidents are passed to `FixedSizeList` from `react-window` to render only visible DOM nodes,

maintaining 60 FPS scrolling.

4. Incidents with `severity == 'critical'` display pulsing red UI borders and sound an optional dispatcher chime.
5. Google Maps container initializes `MarkerClusterer`, aggregating proximate pings into cluster badges to preserve GPU rendering performance.
6. Commander clicks "Acknowledge" on an incident card; client updates Firestore document status to `acknowledged`.
7. `onSOSUpdate` trigger fires in Cloud Functions, generating an immutable Web3 snapshot for state transition auditing.

4.4 Acceptance Criteria (Given-When-Then)

```
Scenario: Real-time incident receipt without page refresh
  Given a police dispatcher has the Web Command Center open on the SOS Incident Queue
  When a pilgrim triggers an SOS alert in the field
  Then the new incident card shall appear at the top of the queue within 2.0 seconds
  And clicking "Acknowledge" shall update the document status in Firestore
```

5. Feature Specification 4: Web3 Cryptographic Trust Layer Anchoring

5.1 Metadata & Description

- **Requirement ID:** `FR-W3-004`
- **Module:** Trust & Audit Subsystem
- **Feature:** Asynchronous 0-ETH Event Snapshot Hashing and Testnet Anchoring
- **Description:** Asynchronously captures canonical JSON snapshots of critical emergency lifecycle events, generates a SHA-256 hash, and embeds it into the hex data payload of a 0-ETH Ethereum transaction on Sepolia testnet to guarantee immutable public auditability.
- **Implementation Source:** `functions/src/trustLayer/blockchainService.ts`, `functions/src/trustLayer/hashUtils.ts`, `functions/src/index.ts:237-312`

5.2 Execution Flow & Fail-Safe Architecture

```
Cloud Function Trigger (onSOSCreate / onSOSUpdate)
       │
       ▼
```

```
1. Extract Event Snapshot (Canonical JSON: entityId, status, severity, timestamp)
        │
        ▼
2. Generate SHA-256 Hash (generateEventHash)
        │
        ▼
3. Call anchorHash(canonicalHash)
        │
        ├──────────────────────────────────────┐
    ┌───┴──────────────────────────────────┐   │
    │ Check BLOCKCHAIN_ENABLED & RPC       │   │
    ▼                                      ▼   │
[RPC Online]                        [RPC Offline / Disabled]
Sign 0 ETH Self-Tx                  Return status: 'LOCAL_PROOF_ONLY'
data: "0x" + canonicalHash          transactionHash: null
Broadcast to Sepolia                Log fallback state
    │                                       │
    └──────────────────┬────────────────────┘
                       ▼
4. Write record to Firestore /blockchainProofs/
   (canonicalHash, transactionHash, anchoringStatus, snapshot)
```

5.3 Acceptance Criteria (Given-When-Then)

```
Scenario: Cryptographic anchoring of SOS creation
  Given a new emergency alert is created in sosAlerts
  When the onSOSCreate background trigger executes
  Then it shall generate a canonical SHA-256 hash of the alert state
  And it shall invoke anchorHash to anchor the hash into a blockchain transaction
  And a document shall be written to blockchainProofs containing canonicalHash and anchoringStatus
```

6. Functional Traceability Summary

All 4 major functional modules detailed above directly implement the business requirements defined in `06_BRD/BRD.md`, satisfy IEEE 829 test cases in `IEEE_TEST_CASES.md`, and are traceable to executable code in `functions/src/index.ts` and `mobile/lib/`.